

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное образовательное учреждение высшего
образования
«Санкт-Петербургский государственный морской технический университет»
ФАКУЛЬТЕТ ЦИФРОВЫХ ПРОМЫШЛЕННЫХ ТЕХНОЛОГИЙ
Кафедра Киберфизических систем

Курсовая работа по теме:
Хеш-таблица: проектирование, реализация и сравнительный анализ
методов разрешения коллизий

Выполнил студент
группы: 20121
Ефремов Н. Р.

Проверил:
Ложкин В. А.

Санкт-Петербург
2026

Содержание

Введение	3
1 Теоретическая часть	4
1.1 Понятие хеш-таблицы	4
1.2 Хеш-функции	4
1.3 Коэффициент загрузки и перехеширование	5
1.4 Методы разрешения коллизий	6
2 Практическая часть	7
2.1 Архитектура программы	7
2.2 Описание интерфейсов	8
2.3 Реализация ключевых алгоритмов	9
2.4 Управление памятью	12
3 Тестирование и анализ производительности	13
3.1 Юнит-тесты	13
3.2 Результаты бенчмарков	15
3.3 Влияние коэффициента загрузки	21
3.4 Анализ результатов	25
Заключение	27
Список использованных источников	28
Приложение	29

Введение

Хеш-таблицы - одна из самых востребованных структур данных [1, с. 223, п. 12]. Они обеспечивают в среднем константное время для операций вставки, поиска и удаления. Ключом может быть любое значение, для которого определена стабильная и равномерно распределяющая хеш-функция. Благодаря этому хеш-таблицы широко применяются в самых разных задачах, а в некоторых языках программирования (например, объекты в JavaScript или словари в Python) они служат базовым контейнером.

Хотя среднее время работы обычно считается постоянным, реальная производительность сильно зависит от конкретной реализации - в частности, от способа разрешения коллизий. В данной работе рассматриваются три метода: метод цепочек (Separate Chaining), линейное и квадратичное пробирование (оба относятся к открытой адресации).

Цель работы - спроектировать и реализовать на C++ обобщённую хеш-таблицу, а затем сравнить производительность трёх методов, чтобы выбрать оптимальный.

Язык C++ выбран из-за высокой производительности, управления памятью и мощных возможностей метапрограммирования [3, с. 204, г. 8], позволяющих реализовать эффективную структуру без потерь на этапе исполнения программы. Для хранения данных используются стандартные контейнеры (`std::vector`, `std::forward_list`), что гарантирует корректное управление памятью.

1 Теоретическая часть

1.1 Понятие хеш-таблицы

Хеш-таблица - это массив, элементы которого называют корзинами или слотами. Индекс для хранения элемента вычисляется по ключу с помощью хеш-функции. Идеальная хеш-функция распределяет ключи по индексам равномерно, что даёт доступ за $O(1)$ в среднем. Однако на практике коллизии (когда разным ключам соответствует один индекс) неизбежны, поэтому необходимы механизмы их разрешения [5, с. 257, п. 5.3].

Основные операции:

- Вставка: вычислить хеш, найти свободное место и поместить туда ключ со значением.
- Поиск: вычислить хеш, пройти по цепочке коллизий до искомого ключа и вернуть значение.
- Удаление: вычислить хеш, найти ключ и пометить его как удалённый (при открытой адресации) или удалить из списка (при цепочках).

1.2 Хеш-функции

Хеш-функция должна быть детерминированной, быстрой и обеспечивать равномерное распределение [5, с. 257, п. 5.2]. Для целых чисел часто используют остаток от деления на размер таблицы - эта операция выполняется внутри таблицы, поэтому хеш-функция сама не занимается приведением к индексу. Для знаковых чисел нужно корректно обрабатывать знак. Например, можно использовать такое преобразование:

Листинг 1 — Хеш-функция для целых чисел со знаком

```
// 0 => 0, 1 => 2, -1 => 1, 2 => 4, -2 => 3
size_t hash(int key) {
    return static_cast<size_t>(
        key >= 0
        ? 2 * key
        : 2 * (-key) - 1
    );
}
```

Для строк применяют более сложные алгоритмы, например, DJB2 - он даёт хорошее распределение и прост в реализации:

Листинг 2 — Хеш-функция DJB2 для строк

```
const size_t DJB2_START = 5381;
size_t hash(const std::string &str) {
    size_t hash = DJB2_START;
    for (char ch : str) {
        hash = ((hash << 5) + hash) + static_cast<size_t>(ch);
    }
    return hash;
}
```

1.3 Коэффициент загрузки и перехеширование

Коэффициент загрузки $\alpha = \frac{N}{M}$, где N - число активных элементов, M - размер массива. Как и в динамических массивах, размер таблицы может быть больше числа элементов, чтобы оставить запас для будущих вставок. Когда α превышает заданный порог (по умолчанию 0.5 для открытой адресации и 1.0 для цепочек), выполняется перехеширование - создаётся таблица большего размера, и все элементы переносятся заново. Это позволяет сохранять среднее время доступа близким к константе. Аналогично, при сильном уменьшении числа элементов таблица может быть сжата для экономии памяти.

1.4 Методы разрешения коллизий

1.4.1 Метод цепочек (Separate Chaining)

В этом подходе каждая ячейка таблицы является головой связанного списка (или другого контейнера), в котором хранятся все элементы с одинаковым хешем. Вставка добавляет новый элемент в начало списка (если ключ уже есть, значение обновляется). Поиск требует обхода списка до нахождения нужного ключа, удаление - удаления узла из списка [2, с. 578].

Преимущества: простота, устойчивость к высокому коэффициенту загрузки (коллизии влияют только на конкретные списки, не затрагивая остальную таблицу). Недостатки: дополнительные затраты на работу с указателями.

1.4.2 Открытая адресация (Open Addressing)

Здесь все элементы хранятся непосредственно в массиве. При коллизии используется пробирование - последовательный поиск свободной ячейки по определённому правилу [5, с. 258, п. 5.3]. Удаление помечает элемент специальным маркером, чтобы не нарушать цепочки пробирования; окончательно записи удаляются только при перехэшировании.

Линейное пробирование: шаг постоянен: $h(k, i) = (h(k) + i) \bmod m$. Просто, но приводит к первичной кластеризации - скоплению занятых ячеек, что замедляет поиск.

Квадратичное пробирование: шаг растёт квадратично: $h(k, i) = (h(k) + c_1 i + c_2 i^2) \bmod m$. Кластеризация уменьшается, но не гарантируется нахождение свободной ячейки, если таблица почти заполнена.

В работе реализованы оба метода. Размер таблицы всегда выбирается как степень двойки, что позволяет заменить операцию взятия по модулю на побитовое И (& (size-1)) - это ускоряет вычисления (широко известная оптимизация, применяемая практически во всех имплементациях хеш-таблиц).

2 Практическая часть

2.1 Архитектура программы

Программа состоит из следующих компонентов:

- Структура `Hash<T>` и её специализация для `std::string` - определяют хеш-функцию для ключей.
- Шаблонный класс `Entry<Key, Value>` - хранит ключ, значение и флаги состояния (активен/удалён). В нём можно было бы также кешировать исходный хеш, но в данной реализации это не сделано.
- Классы стратегий разрешения коллизий: `ChainingStrategy` и `OpenAddressingStrategy` (с параметром стратегии пробирования).
- Шаблонный класс `OrderedHashTable<Key, Value, CollisionStrategy, Hash>` - основной интерфейс для пользователя. Он использует заданную стратегию и хранит массив записей `Entry`, сохраняя порядок вставки.

UML-диаграмма классов приведена на Рисунке 1.

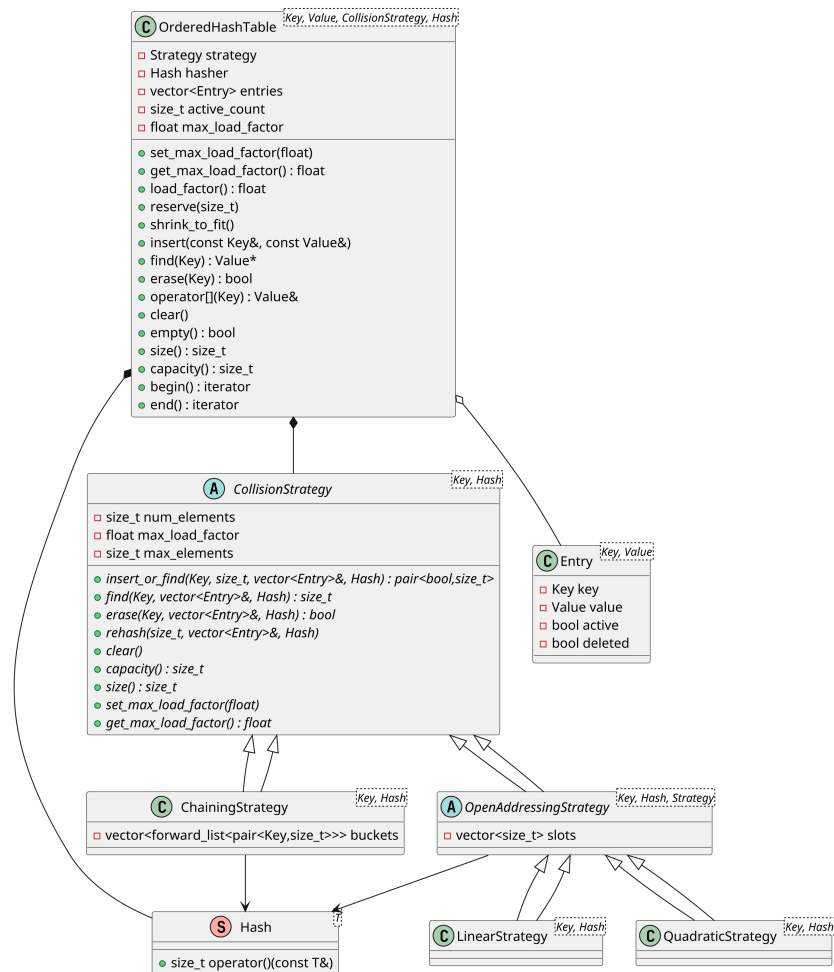


Рисунок 1. UML-диаграмма классов

2.2 Описание интерфейсов

Стратегии разрешения коллизий предоставляют методы, необходимые для работы таблицы:

- `insert_or_find(key, new_index, entries, hasher)` - пытается вставить ключ, возвращает индекс, если ключ уже существует.
- `find(key, entries, hasher)` - ищет ключ и возвращает его индекс.
- `erase(key, entries, hasher)` - удаляет ключ.
- `rehash(new_size, entries, hasher)` - перестраивает таблицу под новый размер.
- `clear()`, `capacity()`, `size()` - стандартные методы для управления размером.

Класс `OrderedHashTable` использует эти методы и предоставляет публичный интерфейс:

- `insert(key, value)` - вставка или обновление значения.
- `find(key)` - поиск, возвращает указатель на значение или `nullptr`.
- `erase(key)` - удаление по ключу.
- `operator[] (key)` - доступ по ключу с автоматической вставкой значения по умолчанию.
- `clear()`, `empty()`, `size()`, `capacity()` - стандартные методы.
- `set_max_load_factor(float)` и `get_max_load_factor()` - управление порогом перехеширования.
- `reserve(count)` - предварительное выделение памяти.
- `shrink_to_fit()` - уменьшение таблицы при низкой загрузке.
- `begin()`, `end()` - итераторы для обхода активных элементов (удалённые пропускаются).

2.3 Реализация ключевых алгоритмов

Блок-схема вставки (обобщённая для всех стратегий) показана на Рисунке 2. Алгоритм включает вычисление хеша, поиск ключа, вставку в свободную позицию и при необходимости перехэширование.

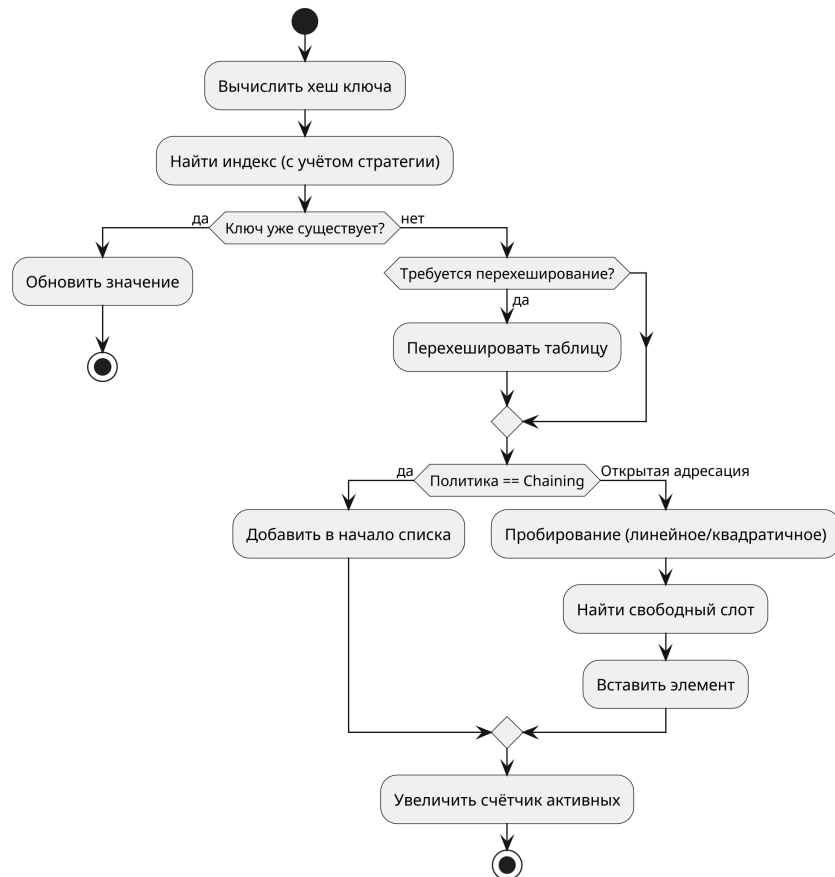


Рисунок 2. Алгоритм вставки элемента

Более детально алгоритмы вычисления индекса для каждого метода представлены на следующих рисунках. На Рисунок 3 показан общий подход к вычислению начального индекса, а на Рисунок 4 и Рисунок 5 - соответственно линейное и квадратичное пробирование.

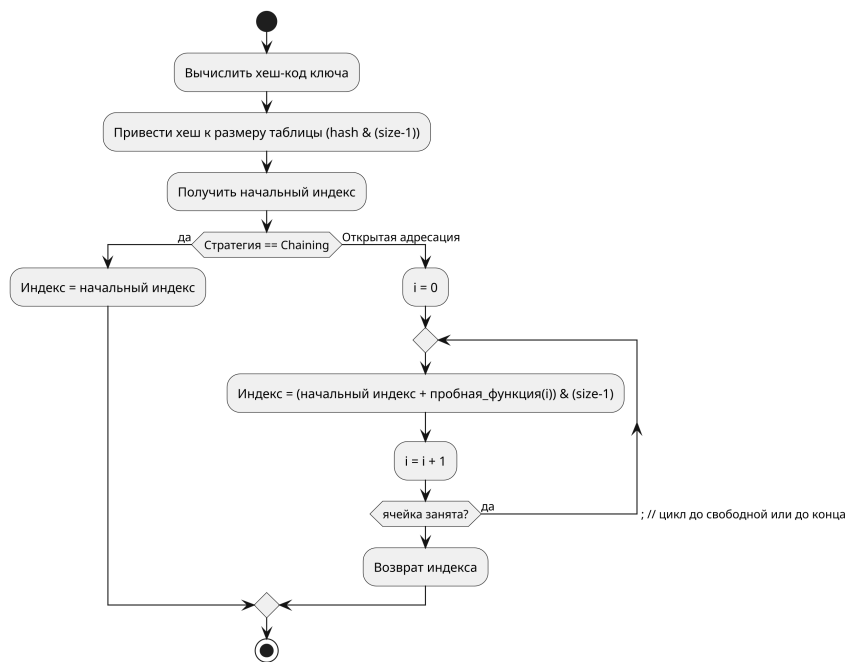


Рисунок 3. Алгоритм вычисления индекса

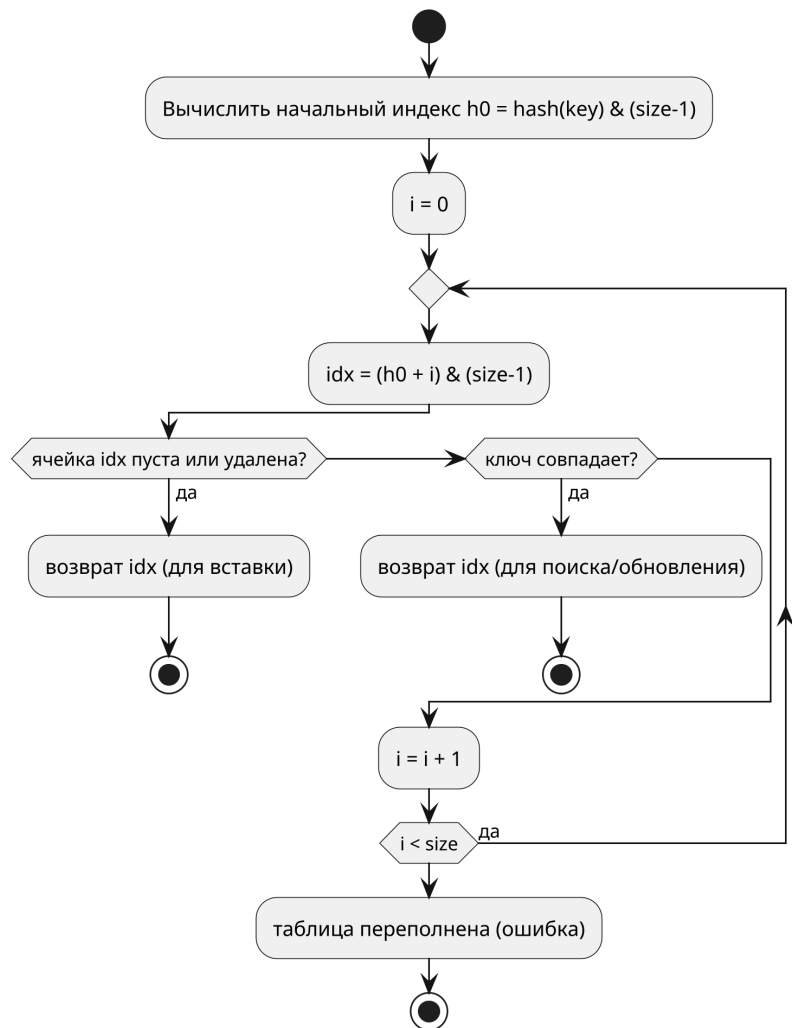


Рисунок 4. Алгоритм линейного пробирования

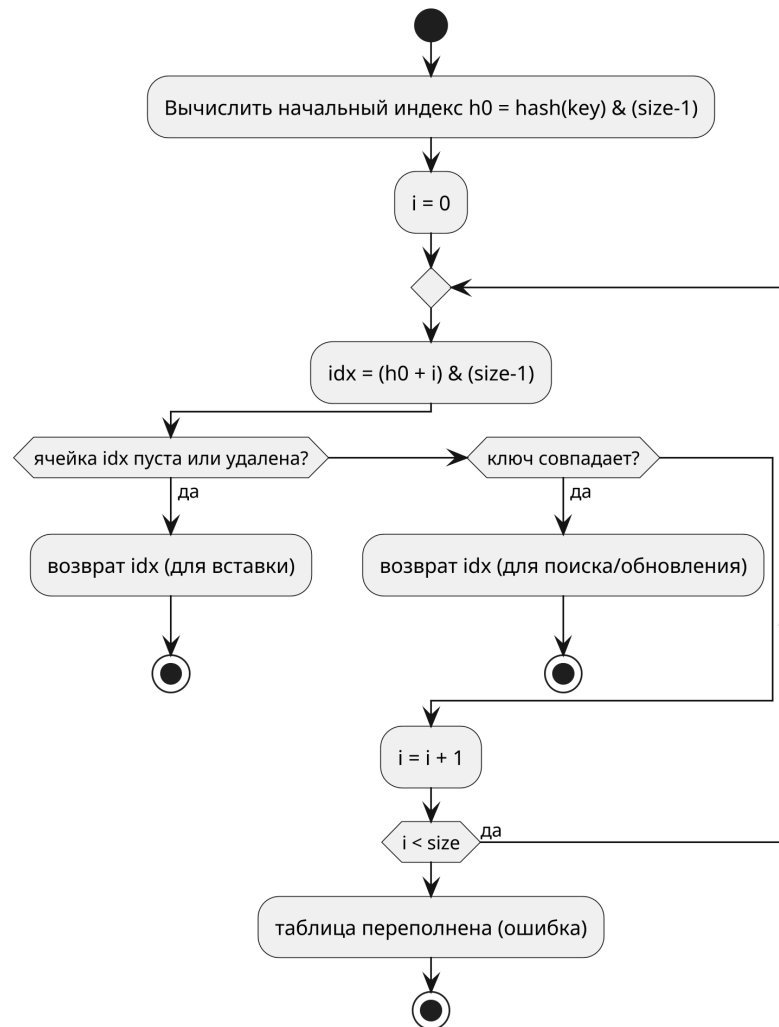


Рисунок 5. Алгоритм квадратичного пробирования

В методе цепочек вставка добавляет элемент в начало списка; в открытой адресации выполняется пробирование с шагом, определяемым стратегией. Удаление в цепочках удаляет узел из списка; в открытой адресации - устанавливает маркер удаления.

Перехеширование создаёт новый массив (или списки) и заново вставляет все активные элементы с использованием текущей хеш-функции.

Поиск и удаление следуют аналогичной логике: по хешу и правилам пробирования находится запись, затем возвращается значение или она помечается как удалённая.

2.4 Управление памятью

Вся динамическая память управляется стандартными контейнерами (`std::vector`, `std::forward_list`), которые автоматически освобождают ресурсы при разрушении. В коде нет явных вызовов `new/delete`, что исключает утечки при корректном использовании. Проверка с помощью Valgrind не обнаружила утечек.

3 Тестирование и анализ производительности

3.1 Юнит-тесты

Для проверки корректности реализован расширенный набор юнит-тестов, покрывающих как базовые, так и продвинутые сценарии использования хеш-таблицы. Тесты запускаются для всех трёх стратегий (Chaining, Linear, Quadratic) и включают:

- вставку и поиск существующих/отсутствующих ключей;
- обновление существующего ключа;
- удаление и повторное удаление;
- работу оператора [];
- соблюдение коэффициента загрузки при массовой вставке;
- итерацию по элементам (включая константный итератор);
- работу с константной таблицей и строковыми ключами;
- вставку и проверку большого числа элементов (5000);
- сохранение порядка вставки после удалений и перехеширований;
- уменьшение ёмкости (shrink_to_fit);
- предварительное резервирование памяти (reserve);
- очистку таблицы (clear);
- конструирование на месте (emplace);
- семантику перемещения (move semantics);
- смешанные операции (вставка, обновление, удаление);
- корректность итераторов после удаления части элементов;
- работу с граничными коэффициентами загрузки;
- обработку намеренных коллизий (ключи, хеширующиеся в один слот).

Все тесты успешно выполняются. Ниже приведён вывод тестового прогона для всех трёх стратегий:

```

==664371== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==664371== Command: ./build/tests
==664371==
== Testing ChainingStrategy ==
test_insert_and_find... PASS (24457 µs)
test_update_existing... PASS (1461 µs)
test_erase... PASS (10534 µs)
test_operator_bracket... PASS (1802 µs)
test_load_factor_and_rehash... PASS (14449 µs)
test_iteration... PASS (4712 µs)
test_const_find... PASS (1294 µs)
test_string_keys... PASS (32699 µs)
test_large_insert_find... PASS (161953 µs)
test_order_preservation... PASS (12586 µs)
test_shrink_to_fit... PASS (35706 µs)
test_reserve... PASS (9957 µs)
test_clear... PASS (5914 µs)
test_emplace... PASS (9667 µs)
test_move_semantics... PASS (12055 µs)
test_mixed_operations... PASS (8749 µs)
test_iterator_after_delete... PASS (3329 µs)
test_const_iteration... PASS (3287 µs)
test_edge_load_factors... PASS (8245 µs)
test_collision_handling... PASS (5127 µs)
Total ChainingStrategy time: 373894 µs

== Testing LinearStrategy ==
test_insert_and_find... PASS (11813 µs)
test_update_existing... PASS (1432 µs)
test_erase... PASS (7460 µs)
test_operator_bracket... PASS (1898 µs)
test_load_factor_and_rehash... PASS (5117 µs)
test_iteration... PASS (3711 µs)
test_const_find... PASS (1052 µs)
test_string_keys... PASS (6563 µs)
test_large_insert_find... PASS (42808 µs)
test_order_preservation... PASS (5026 µs)
test_shrink_to_fit... PASS (12258 µs)
test_reserve... PASS (4240 µs)
test_clear... PASS (3802 µs)
test_emplace... PASS (4690 µs)
test_move_semantics... PASS (5366 µs)
test_mixed_operations... PASS (4852 µs)
test_iterator_after_delete... PASS (2812 µs)
test_const_iteration... PASS (3196 µs)
test_edge_load_factors... PASS (3063 µs)
test_collision_handling... PASS (3647 µs)
Total LinearStrategy time: 137104 µs

== Testing QuadraticStrategy ==
test_insert_and_find... PASS (5756 µs)
test_update_existing... PASS (1462 µs)
test_erase... PASS (5818 µs)
test_operator_bracket... PASS (1784 µs)
test_load_factor_and_rehash... PASS (4533 µs)
test_iteration... PASS (3714 µs)
test_const_find... PASS (1017 µs)
test_string_keys... PASS (6299 µs)
test_large_insert_find... PASS (46440 µs)
test_order_preservation... PASS (5299 µs)
test_shrink_to_fit... PASS (12776 µs)
test_reserve... PASS (4259 µs)
test_clear... PASS (2600 µs)
test_emplace... PASS (4592 µs)
test_move_semantics... PASS (5319 µs)
test_mixed_operations... PASS (4754 µs)
test_iterator_after_delete... PASS (2812 µs)
test_const_iteration... PASS (3283 µs)
test_edge_load_factors... PASS (3117 µs)
test_collision_handling... PASS (3664 µs)
Total QuadraticStrategy time: 131623 µs

== Overall result ==
ALL TESTS PASSED (total time: 669576 µs)
==664371==
==664371== HEAP SUMMARY:
==664371==    in use at exit: 0 bytes in 0 blocks
==664371== total heap usage: 18,113 allocs, 18,113 frees, 2,162,728 bytes allocated

```

Рисунок 6. Результаты юнит-тестов для всех стратегий разрешения

3.2 Результаты бенчмарков

Для каждой стратегии использовался стандартный коэффициент загрузки. Измерялось среднее время операций при разных размерах таблицы [5, с. 261, п. 5.4].

3.2.1 Результаты для последовательных ключей

Таблица 1 — Время вставки (sequential)

Политика	N	Среднее (нс)	σ	Мин	Макс
Chaining	1000	71.70	12.99	66.92	132.07
Linear	1000	6.17	0.02	6.13	6.27
Quadratic	1000	7.05	0.37	6.97	10.69
Chaining	10000	114.48	1.41	112.45	119.19
Linear	10000	26.35	0.30	25.72	27.15
Quadratic	10000	27.23	0.32	26.64	28.52
Chaining	100000	100.56	1.03	98.90	105.07
Linear	100000	24.85	0.53	24.45	26.53
Quadratic	100000	25.75	0.61	25.27	27.63

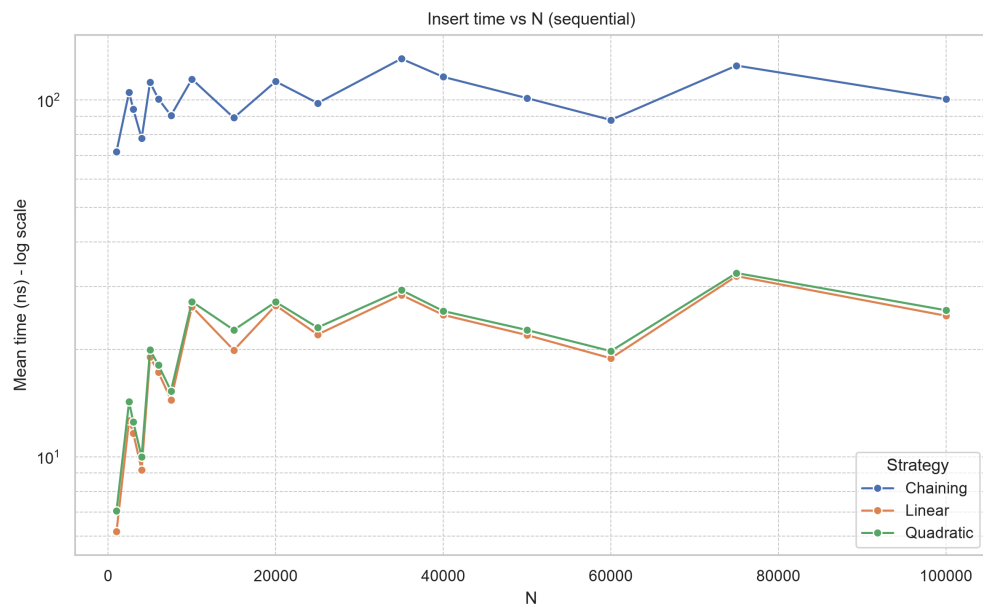


Рисунок 7. Время вставки (последовательные ключи)

Таблица 2 — Время поиска (sequential)

Политика	N	Среднее (нс)	σ	Мин	Макс
Chaining	1000	5.73	1.09	5.31	11.15
Linear	1000	5.88	0.92	5.52	10.14
Quadratic	1000	5.08	0.32	4.94	7.89
Chaining	10000	6.28	0.28	6.05	7.60
Linear	10000	6.81	0.09	6.71	7.19
Quadratic	10000	6.27	0.18	6.01	7.08
Chaining	100000	7.83	0.58	7.55	11.27
Linear	100000	7.45	0.49	7.26	11.20
Quadratic	100000	6.85	0.13	6.73	7.62

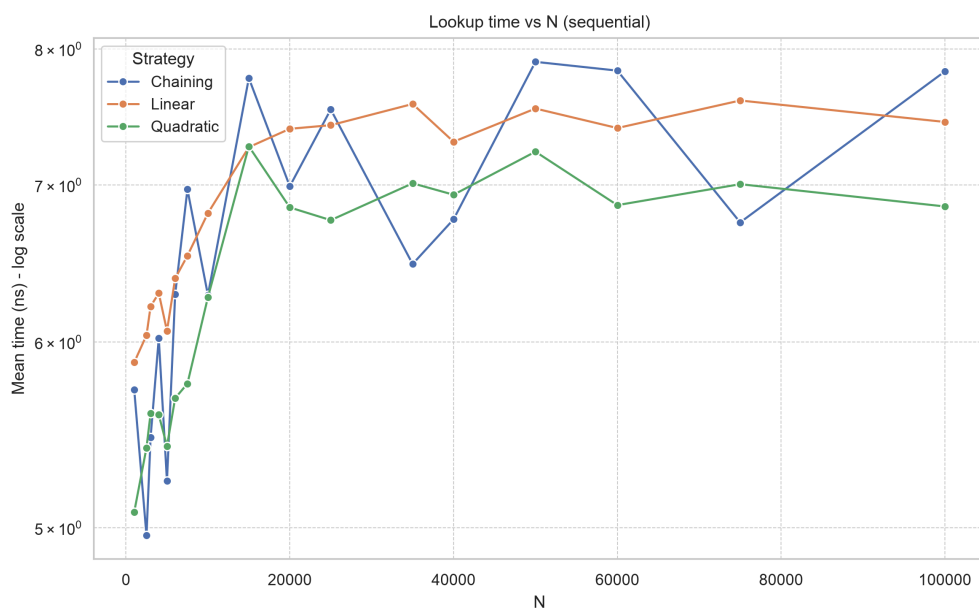


Рисунок 8. Время поиска (последовательные ключи)

3.2.2 Результаты для равномерно распределённых ключей

Таблица 3 — Время вставки (uniform)

Политика	N	Среднее (нс)	σ	Мин	Макс
Chaining	1000	100.03	0.92	99.01	104.09
Linear	1000	7.67	3.87	7.16	45.75
Quadratic	1000	7.84	0.26	7.75	10.45
Chaining	10000	92.42	0.78	91.46	95.41
Linear	10000	14.57	0.22	14.33	15.90
Quadratic	10000	15.29	0.18	15.06	15.98
Chaining	100000	147.55	2.23	144.25	156.71
Linear	100000	33.27	0.87	32.32	35.96
Quadratic	100000	34.15	0.87	33.34	38.48

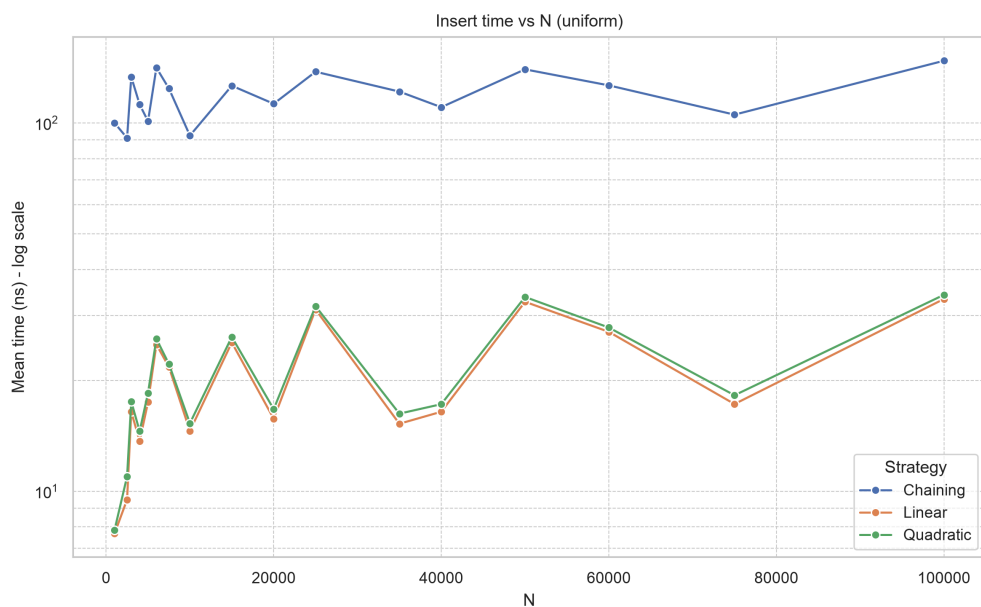


Рисунок 9. Время вставки (равномерные ключи)

Таблица 4 — Время поиска (uniform)

Политика	N	Среднее (нс)	σ	Мин	Макс
Chaining	1000	5.07	0.19	4.92	6.08
Linear	1000	4.63	0.06	4.57	4.99
Quadratic	1000	4.08	0.06	4.03	4.38
Chaining	10000	8.84	0.17	8.63	9.73
Linear	10000	8.44	0.11	8.33	9.02
Quadratic	10000	7.61	0.30	7.29	8.79
Chaining	100000	6.92	0.32	6.73	9.10
Linear	100000	6.74	0.08	6.67	7.18
Quadratic	100000	6.52	0.08	6.45	6.84

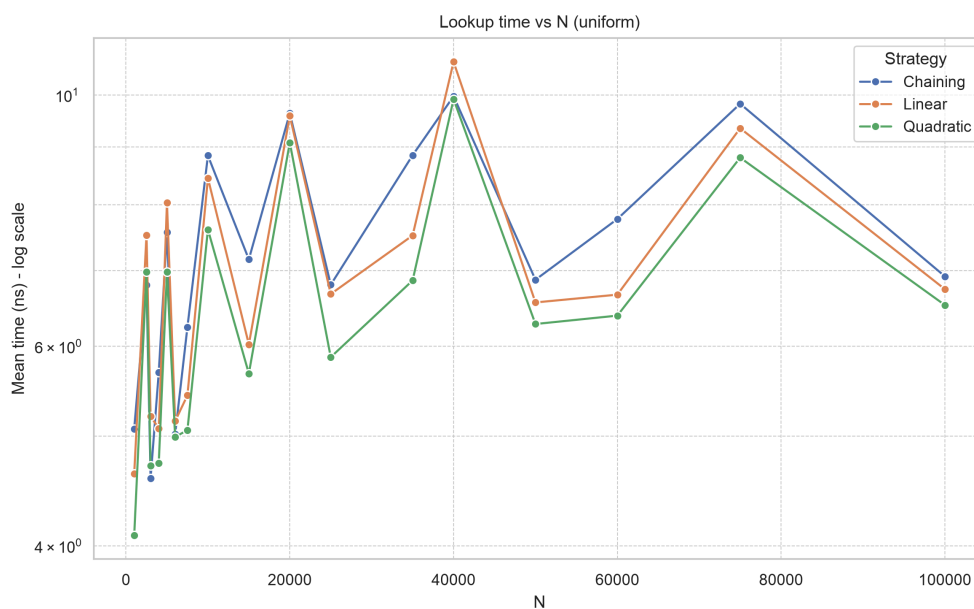


Рисунок 10. Время поиска (равномерные ключи)

3.2.3 Результаты для «плохих» ключей (коллизии)

Таблица 5 — Время вставки (bad)

Политика	N	Среднее (нс)	σ	Мин	Макс
Chaining	1000	1150.87	19.49	1124.74	1222.21
Linear	1000	966.28	12.86	941.15	1033.14
Quadratic	1000	1086.03	37.94	1055.53	1219.85
Chaining	10000	4628.64	121.45	4405.69	5006.78
Linear	10000	1230.11	25.80	1198.97	1296.53
Quadratic	10000	1478.70	32.85	1419.07	1605.92
Chaining	100000	6930.89	49.75	6895.17	7279.31
Linear	100000	1440.32	6.72	1429.94	1478.91
Quadratic	100000	1972.52	16.10	1958.87	2053.12

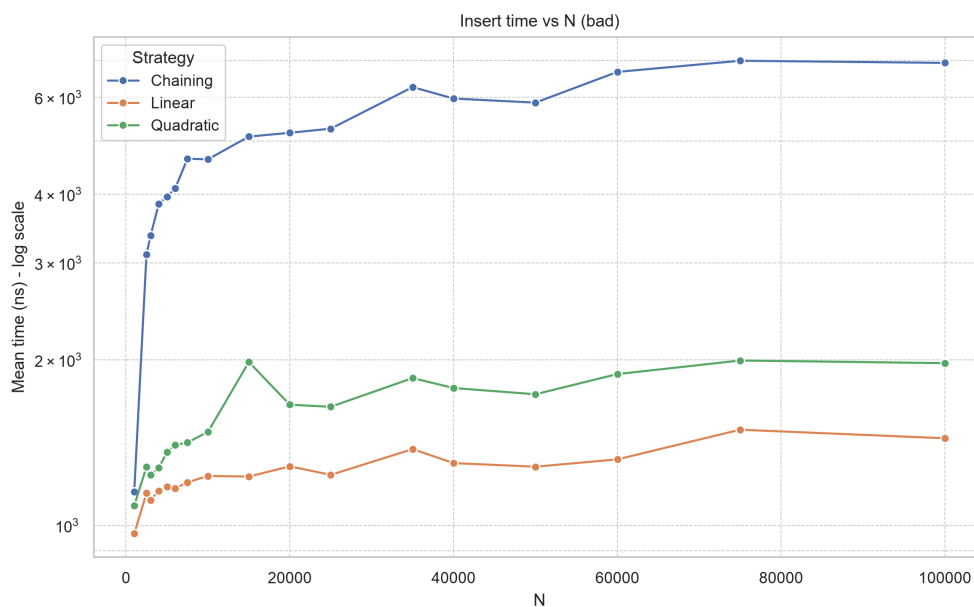


Рисунок 11. Время вставки (ключи с коллизиями)

Таблица 6 — Время поиска (bad)

Политика	N	Среднее (нс)	σ	Мин	Макс
Chaining	1000	276.55	3.31	269.87	287.98
Linear	1000	353.26	27.57	332.43	616.73
Quadratic	1000	237.82	4.65	233.56	253.25
Chaining	10000	576.53	9.54	561.43	618.96
Linear	10000	201.50	5.97	193.64	231.50
Quadratic	10000	197.90	4.39	192.03	219.61
Chaining	100000	1714.08	12.69	1702.79	1802.33
Linear	100000	315.44	6.98	311.68	351.78
Quadratic	100000	364.82	8.54	359.47	418.10

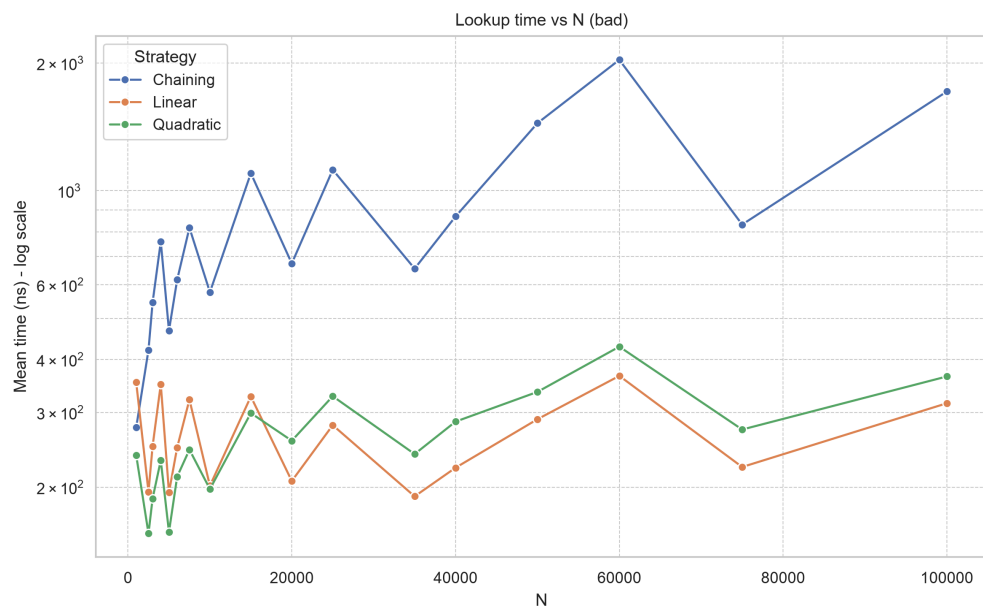


Рисунок 12. Время поиска (ключи с коллизиями)

3.3 Влияние коэффициента загрузки

Коэффициент загрузки - один из ключевых параметров, влияющих на производительность. В этом разделе представлены результаты бенчмарков при разных значениях α для всех трёх стратегий. (Для метода цепочек α может превышать 1, так как каждый слот - это список, а для пробирования это невозможно, поэтому графики обрываются.)

Для каждого распределения ключей и каждого размера таблицы измерялись средние времена вставки и поиска. На графиках ниже показана зависимость времени от α для таблицы из 10000 элементов (для других размеров картина аналогична).

3.3.1 Результаты для последовательных ключей

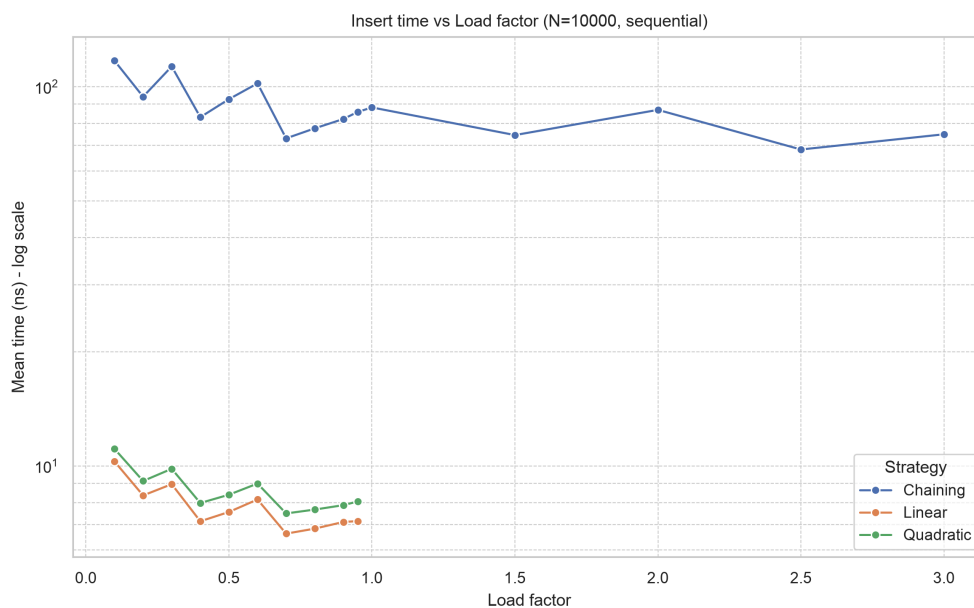


Рисунок 13. Время вставки от load factor (последовательные ключи, N=10000)

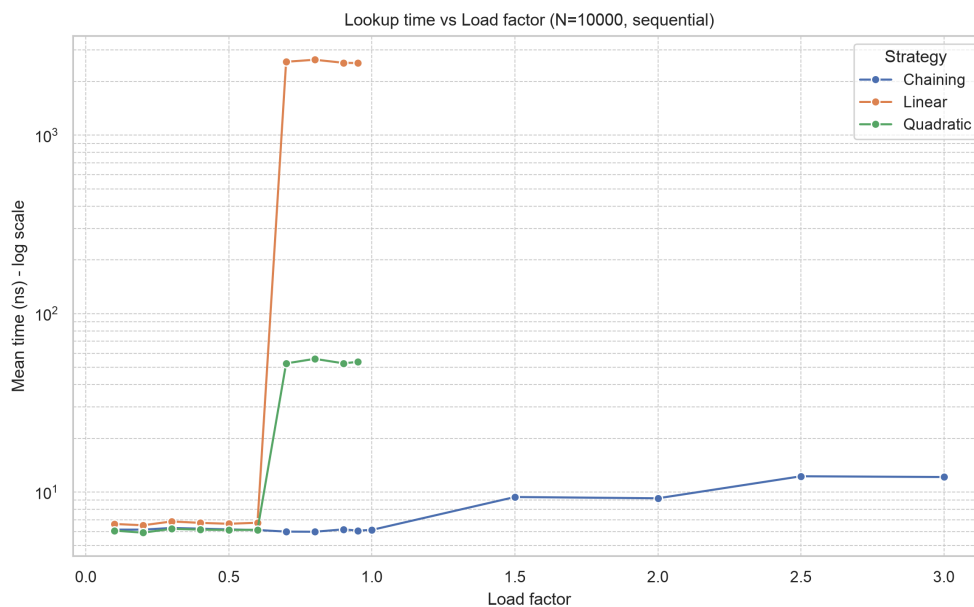


Рисунок 14. Время поиска от load factor (последовательные ключи, N=10000)

3.3.2 Результаты для равномерно распределённых ключей

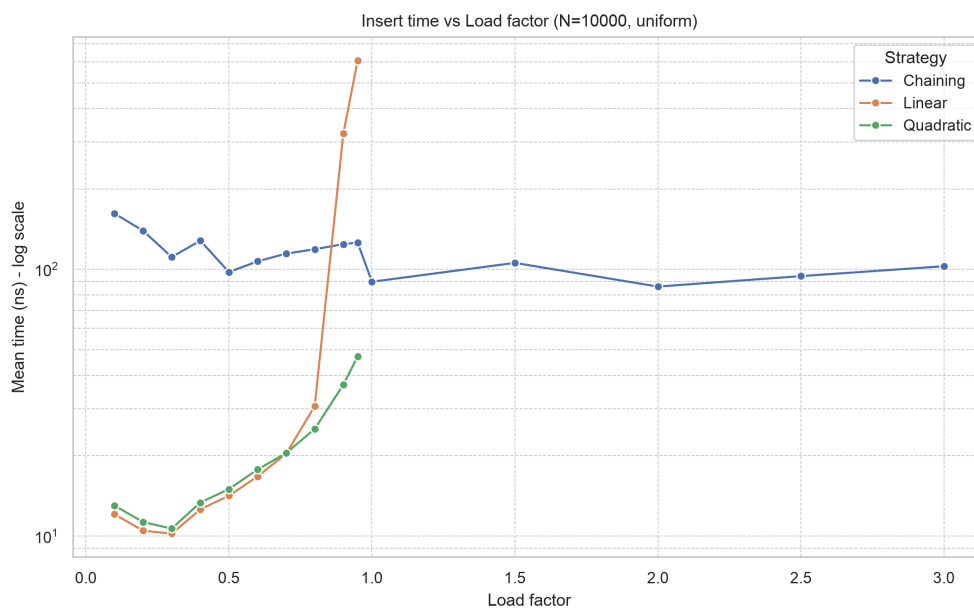


Рисунок 15. Время вставки от load factor (равномерные ключи, N=10000)

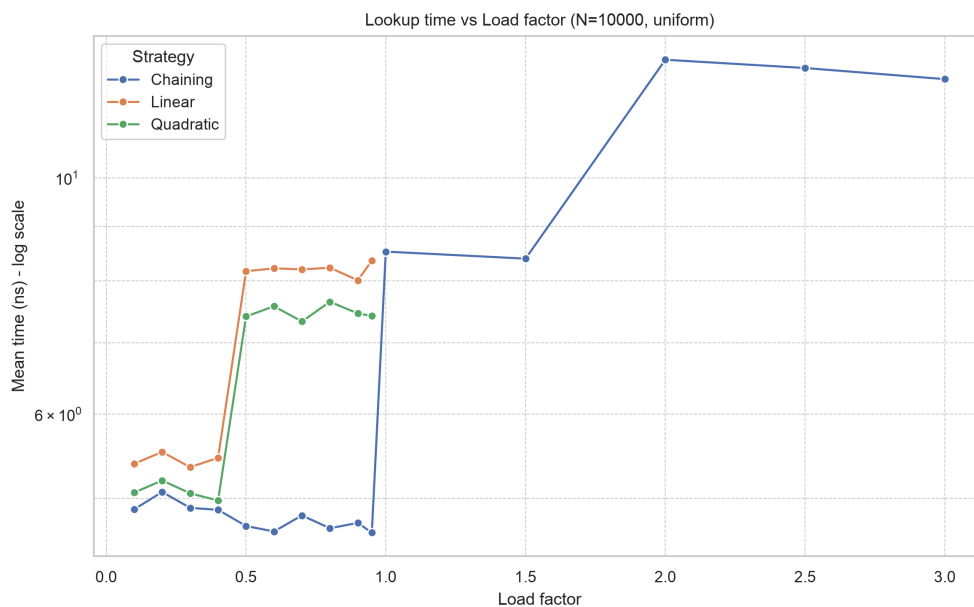


Рисунок 16. Время поиска от load factor (равномерные ключи, N=10000)

3.3.3 Результаты для «плохих» ключей (коллизии)

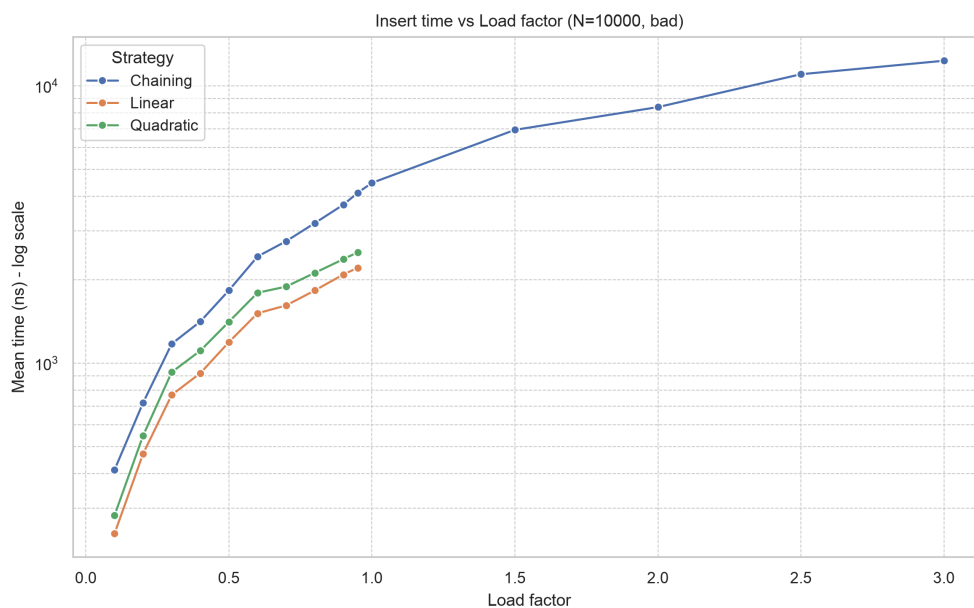


Рисунок 17. Время вставки от load factor (плохие ключи, N=10000)

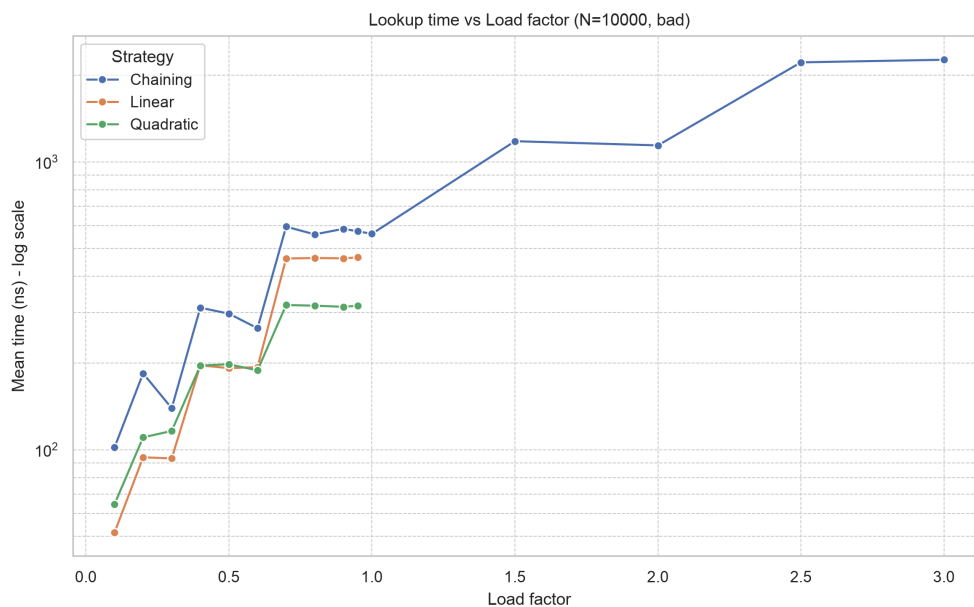


Рисунок 18. Время поиска от load factor (плохие ключи, N=10000)

3.4 Анализ результатов

Анализ производительности трёх стратегий проведён по нескольким измерениям: время вставки и поиска при разных размерах таблицы и распределениях ключей, а также зависимость от коэффициента загрузки.

Вставка: Метод цепочек (Chaining) стабильно медленнее открытой адресации при любых распределениях ключей. Основная причина - каждая вставка требует динамического выделения памяти под новый узел связного списка (`std::forward_list`). Открытая адресация (линейное и квадратичное пробирование) работает с заранее выделенным непрерывным массивом и при попадании в свободную ячейку выполняет лишь запись, что значительно быстрее. При хороших хеш-функциях (последовательные и равномерные ключи) линейное и квадратичное пробирование показывают практически одинаковое время вставки - разница становится заметной только при высокой загрузке или большом количестве коллизий.

Поиск: Картина меняется. При умеренных коэффициентах загрузки ($\alpha \leq 0.5$) и равномерном распределении ключей все три метода показывают близкие результаты, причём Chaining иногда оказывается даже немного быстрее, так как после вычисления индекса корзины остаётся пройти короткий список (обычно 0-1 элемент), в то время как открытая адресация делает несколько проб, каждая из которых может быть кеш-промахом. С ростом α и увеличением числа коллизий открытая адресация деградирует: линейное пробирование страдает от первичной кластеризации, квадратичное - в меньшей степени, но оба метода начинают проигрывать Chaining в поиске при высоких нагрузках. При «плохом» распределении (искусственные коллизии) Chaining в поиске всё же уступает открытой адресации, так как длинные связные списки обходятся медленнее, чем массив с пробами.

Влияние коэффициента загрузки: Для открытой адресации оптимальным компромиссом между используемой памятью и производительностью является $\alpha \approx 0.5$. При более высоких значениях резко возрастает число проб, что ведёт к деградации времени поиска, особенно для линейного пробирования. Квадратичное пробирование значительно устойчивее, но также не рекомендуется к использованию с $\alpha > 0.7$. Chaining, напротив, демонстрирует высокую стабильность: при α от 0.1 до 1.0 время поиска почти не меняется. Это объясняется тем, что коллизии изолированы в отдельных списках и не влияют на соседние

корзины. При $\alpha > 1$ Chaining продолжает работать, однако время операций линейно растёт с длиной списков, особенно при неравномерном хешировании.

Сравнение линейного и квадратичного пробирования: В большинстве реальных сценариев квадратичное пробирование либо не сильно уступает линейному, либо выигрывает. Линейное пробирование склонно к образованию длинных кластеров, что приводит к увеличению среднего числа проб при поиске.

Итог: Квадратичное пробирование выбрано в качестве стратегии по умолчанию, поскольку оно обеспечивает наилучший баланс между скоростью вставки (сравнимой с линейным пробированием), устойчивостью к коллизиям, и скоростью поиска в стандартных ситуациях. Chaining проигрывает во вставке из-за накладных расходов на аллокацию, однако демонстрирует стабильное время поиска при высоких коэффициентах загрузки, что может сделать его хорошим выбором в случае, когда в определённых корзинах может быть множество коллизий (непредсказуемое распределение), но реаллоцировать всю таблицу не хочется из-за ограниченной памяти. Для типичных же сценариев с более-менее равномерно распределёнными хешами, умеренной загрузкой и перемежающимися операциями вставки/поиска, квадратичное пробирование является оптимальным выбором.

Заключение

В ходе работы спроектирована и реализована на C++ хеш-таблица с поддержкой трёх методов разрешения коллизий: цепочки, линейное и квадратичное пробирование. Проведён сравнительный анализ производительности, позволивший обоснованно выбрать оптимальные параметры. Код проверен на утечки памяти с помощью Valgrind и работает корректно. В процессе выполнения получены углублённые знания о хеш-таблицах, улучшены навыки шаблонного метапрограммирования на C++, а также отработаны приёмы автоматической генерации таблиц и графиков для отчётов.

Список использованных источников

1. Кормен Т. Х., Лейзерсон Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ.
2. Седжвик Р. Фундаментальные алгоритмы на C++.
3. Страуструп Б. Язык программирования C++. 2-е изд.
4. Ахо А. В., Хопкрофт Д. Э., Ульман Д. Д. Структуры данных и алгоритмы.
5. Вирт Н. Алгоритмы и структуры данных.
6. Кнут Д. Э. Искусство программирования. Том 3. Сортировка и поиск.
7. cppreference.com. `std::unordered_map`. URL: https://en.cppreference.com/w/cpp/container/unordered_map (дата обращения: 23.06.2026).
8. cppreference.com. `std::hash`. URL: <https://en.cppreference.com/w/cpp/utility/hash> (дата обращения: 23.06.2026).
9. cppreference.com. `std::vector`. URL: <https://en.cppreference.com/w/cpp/container/vector> (дата обращения: 23.06.2026).

Приложение

- Репозиторий с исходным кодом и скриптами сборки данного документа:
<https://github.com/lambda-abstraction/hashtable>